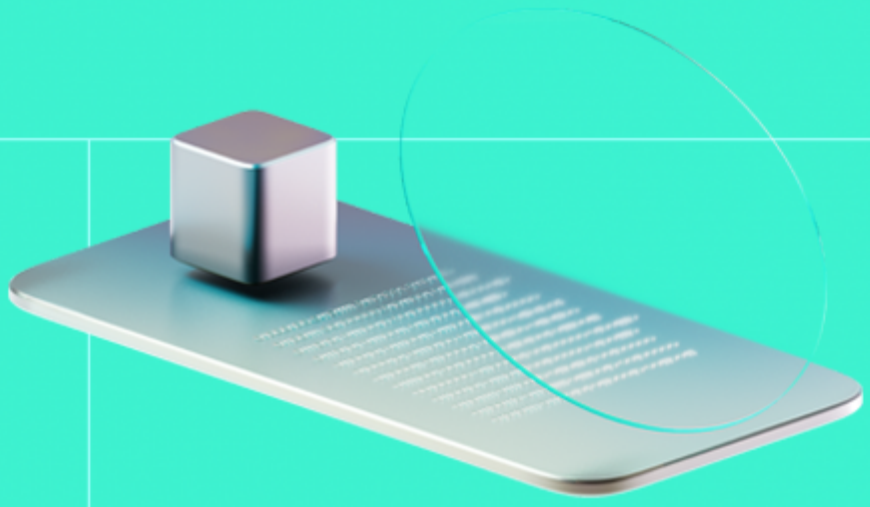
HACKEN

# Smart Contract Code Review And Security Analysis Report



**Customer:** Dlicom

**Date:** 13/02/2026

We express our gratitude to the Dlicom team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Dlicom is an ERC20 token with minting and burning capability. It also includes a switch for a marketLaunch.

## Document

| | |
|---|---|
| Name | Smart Contract Code Review and Security Analysis Report for Dlicom |
| Audited By | David Camps Novi |
| Approved By | Olesia Bilenka |
| Website | https://dlicom.io/ |
| Changelog | 06/02/2026 - Preliminary Report |
| | 13/02/2026 - Final Report |
| Platform | Base |
| Language | Solidity |
| Tags | Fungible Token; Centralization; Upgradable |
| Methodology | https://docs.hacken.io/methodologies/smart-contracts |

## Review Scope

| | |
|---|---|
| Repository | https://github.com/spl4bs/dlicom-token-sc |
| Commit | c1bd927 |

## Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

| 3 | 0 | 3 | 0 |
|---|---|---|---|
| Total Findings | Resolved | Accepted | Mitigated |

## Findings by Severity

| Severity | Count |
|---|---|
| Critical | 0 |
| High | 0 |
| Medium | 1 |
| Low | 0 |

| Vulnerability | Severity | Status |
|---|---|---|
| F-2026-14957 - Overprivileged Token Burning Capacity | Medium | Accepted |
| F-2026-14951 - Floating Pragma | Info | Accepted |
| F-2026-14952 - Missing _disableInitializers() in Upgradable Contract Constructor | Info | Accepted |

## Documentation quality

- Functional requirements are provided via NatSpec but no further system description and requirements.
- Technical description is missing.

## Code quality

- The development environment is configured.

## Test coverage

Code coverage of the project is **0%** (branch coverage).

- No tests were provided

# Table of Contents

## System Overview

Dlicom is an ERC20 token with minting and burning capability. It also includes a switch for a marketLaunch.

The system consists of the following contracts:

- **DLICOM** - ERC20 with minting and burning capability, which also includes roles via `AccessControlUpgradeable`
- **AccessControlUpgradeable** - contract to manage the system roles of the DLICOM token contract.

## Privileged roles

- **DEFAULT_ADMIN_ROLE** - can setup the rest of the system roles via `AccessControlUpgradeable`, use `burnSupplyFrom()` to burn tokens from any address and decrease the token `MAX_SUPPLY` and turn on the `marketLaunch`.
- **MINTER_ROLE** - can mint tokens to any given address, limited to reaching the `MAX_SUPPLY` of the token.

## Potential Risks

- The project's contracts are upgradable, allowing the administrator to update the contract logic at any time. While this provides flexibility in addressing issues and evolving the project, it also introduces risks if upgrade processes are not properly managed or secured, potentially allowing for unauthorized changes that could compromise the project's integrity and security.
- The system implements a custom `AccessControlUpgradeable` contract version, which updated the `public` visibility of `hasRole()` to `internal`. It should be noted that performing this change will limit the capacity for any external source to retrieve the required information since will fail to be accessible via `ABI`. This may also result in failed compatibility with other systems expecting to be able to reach the result of `hasRole()` in order to get roles information. At the same time, changing `hasRole()` visibility to `internal` is not serving the purpose of hiding or protecting the roles data, since it can still be obtained.

# Findings

## Vulnerability Details

### F-2026-14957 - Overprivileged Token Burning Capacity - Medium

**Description:**

The system includes two methods that allow non-holder addresses to burn tokens from users:

```
    function burnFrom(address account, uint256 value) public virtual {
        _spendAllowance(account, _msgSender(), value);
        _burn(account, value);
    }

    function burnSupplyFrom(address account, uint256 amount) public onlyRole(
DEFAULT_ADMIN_ROLE) {
        _burn(account, amount);
        MAX_SUPPLY -= amount;
        emit BurnSupplyFrom(account, amount, MAX_SUPPLY);
    }
```

In the first method `burnFrom()` any address with sufficient `allowance` is capable to burn tokens from the given user `account`.

In the second method `burnSupplyFrom()`, the `DEFAULT_ADMIN_ROLE` is capable to burn any amount of tokens from any user, at will, without any restriction or previous requirement from the users.

**Assets:**

- DLICom.sol [https://github.com/spl4bs/dlicom-token-sc]

**Status:** Accepted

#### Classification

**Impact Rate:** 5/5

**Likelihood Rate:** 3/5

**Exploitability:** Dependent

**Complexity:** Simple

**Severity:** Medium

### Recommendations

**Remediation:** It is recommended to limit the capacity of burning tokens from other users by introducing a pre-request mechanism in which a user can first perform a request for their tokens to be burned. This is valid for both `burnFrom()` and `burnSupplyFrom()` methods.

If the intention of the method `burnSupplyFrom()` is to reduce the token's `MAX_SUPPLY`, consider applying the token burning to the same `DEFAULT_ADMIN_ROLE` holder/caller.

**Resolution:** **Accepted**

The described behavior remains present. The `burnFrom()` function permits any address with sufficient allowance to burn assets from another account. The `burnSupplyFrom()` function enables an address holding `DEFAULT_ADMIN_ROLE` to **burn assets from any** account without restriction and to reduce `MAX_SUPPLY`.

The client explicitly confirmed that this design is intentional. The separation between `totalSupply` reduction (via holder or allowance-based burns) and `MAX_SUPPLY` reduction (via an administrative function) is deliberate. The ability of the administrator to burn assets from any account has been acknowledged and accepted as part of the protocol's governance model.

No modifications were introduced to restrict or alter this capability. The issue therefore remains unresolved by design and is formally accepted.

**Client Comment:**

> `burn()` and `burnFrom()` are inherited from **ERC20BurnableUpgradeable** and only decrease the token's `totalSupply` by destroying tokens held by the caller (or by an approved spender). They do not and are not intended to change the protocol's hard cap variable `MAX_SUPPLY`; therefore, user burns reduce circulating supply but leave the configured maximum issuance unchanged.
>
> `burnSupplyFrom()` is a separate administrative cap-management function protected by `DEFAULT_ADMIN_ROLE` and is the only mechanism designed to permanently decrease `MAX_SUPPLY`. This separation is intentional: holder-level burns affect supply in circulation (`totalSupply`), while governance/admin-controlled actions are required to modify the protocol-defined cap (`MAX_SUPPLY`).
>
> Expected invariants: after any operation, `totalSupply` decreases only via burns, and `MAX_SUPPLY` can only decrease via `burnSupplyFrom()`; no function increases `MAX_SUPPLY`.

## F-2026-14951 - Floating Pragma - Info

**Description:** In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.xx`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.16`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

**Status:** Accepted

### Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 1/5

**Exploitability:** Dependent

**Complexity:** Simple

**Severity:** Info

### Recommendations

**Remediation:** It is recommended to **lock the pragma version** to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates. For example, instead of using `^0.8.xx`, explicitly define the version with `pragma solidity 0.8.16;`.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: Solidity GitHub rele

ases or Solidity Bugs by Version. Choose a compiler version with a good track record for stability and security.

**Resolution:** **Accepted**

The Solidity contracts continue to use floating pragma declarations (`^0.8.16`). The pragma version has not been locked to a specific compiler release. The client has acknowledged the risks associated with compiler version discrepancies and has accepted the current configuration without changes.

## F-2026-14952 - Missing _disableInitializers() in Upgradable Contract Constructor - Info

**Description:**

OpenZeppelin's `Initializable` pattern supports secure upgradeable contract deployment by isolating initialization logic. For any contract meant to serve as a non-upgradable implementation, it is standard best practice to disable further initializations:

```
function _disableInitializers() internal virtual { ... }
```

This function prevents any subsequent call to `initialize` or similar initializer functions. The OpenZeppelin documentation explicitly recommends this safeguard for any implementation contracts that might be deployed behind a proxy.

In the current codebase `DLICOM` does not invoke `_disableInitializers` in its constructors.

Omitting this call leaves the implementation contracts open to future initialization, which can pose a risk in proxy-based deployments or if inherited by other systems.

**Assets:**

- DLICom.sol [https://github.com/spl4bs/dlicom-token-sc]

**Status:** Accepted

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 5/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

### Recommendations

**Remediation:** Include the following in the constructor to prevent further initialization:

```
constructor() {
    _disableInitializers();
```

```
}
```

**Resolution:** **Accepted**

The implementation contracts do not invoke `_disableInitializers()` in the constructor. This safeguard, recommended by OpenZeppelin for implementation contracts intended for proxy-based deployments, has not been introduced. The client has acknowledged the risk associated with potential future initialization and has accepted the current design without modification.

# Disclaimers

## Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

## Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

# Appendix 1. Definitions

## Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

hknio/severity-formula

| Severity | Description |
|---|---|
| Critical | Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation. |
| High | High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation. |
| Medium | Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category. |
| Low | Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution. |

## Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

# Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

| Scope Details | |
|---|---|
| Repository | https://github.com/spl4bs/dlicom-token-sc |
| Commit | c1bd927 |
| Whitepaper | N/A |
| Requirements | |
| Technical Requirements | |

| Asset | Type |
|---|---|
| AccessControlUpgradeable.sol [https://github.com/spl4bs/dlicom-token-sc] | Smart Contract |
| DLICom.sol [https://github.com/spl4bs/dlicom-token-sc] | Smart Contract |

# Appendix 3. Additional Valuables

## Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

## Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment, and the Penetration Testing Execution Standard (PTES), These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.